Backtracking and Branch & Bound



General Concepts

- Algorithm strategy
  - *Approach to solving a problem*
  - *May combine several approaches*
- Algorithm structure
  - *Iterative ⇒ execute action in loop*
  - *Recursive ⇒ reapply action to subproblem(s)*
- Problem type
  - *Satisfying* ex: finding a solution to a maze *⇒ find any satisfactory solution*
  - *Optimization* ⇒ *find best solutions (vs. cost metric)*

greedy / dynamic

Backtracking

- Suppose you have to make a series of *decisions,* among various *choices,* where
  - *You don't have enough information to know what to choose* So you choose a solution randomly like going to a building and you want to go somewhere but u don't know which path or doors would go to where you need to go
  - *Each decision leads to a new set of choices*
  - *Some sequence of choices (possibly more than one) may be a solution to your problem*
- Backtracking is a methodical way of trying out various sequences of decisions, until you find one that "works" random algorithm



Backtracking Algorithm

- Based on depth-first recursive search was a mcq question in the final
- Approach
  1. *Tests whether solution has been found*
  2. *If found solution, return it*
  3. *Else for each choice that can be made*
     a) Make that choice
     b) Recur
     c) If recursion returns a solution, return it
  4. *If no choices remain, return failure*
- Some times called "search tree"



Backtracking Algorithm – Example

- Find path through maze
  - *Start at beginning of maze*
  - *If at exit, return true*
  - *Else for each step from current location*
    - Recursively find path
    - Return with first successful step
    - Return false if all steps fail



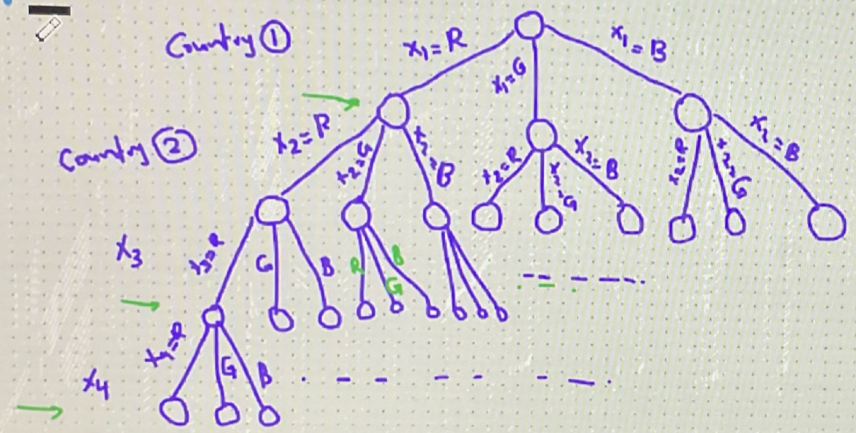
Backtracking Algorithm – Example

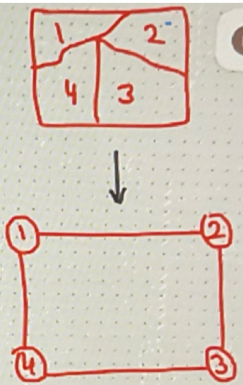
- Color a map with no more than four colors
  - *If all countries have been colored return success*
  - *Else for each color c of four colors and country n*
    - If country n is not adjacent to a country that has been colored c
      - Color country n with color c
      - Recursively color country n+1
      - If successful, return success
  - *Return failure*



Brute Force

state space tree



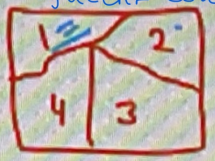


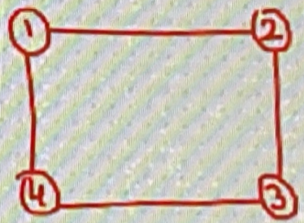
$$1 + 3 + 3 \times 3 + 3 \times 3 \times 3 + 3 \times 3 \times 3 \times 3$$
$$3^0 + 3^1 + 3^2 + 3^3 + 3^4$$

State Space tree using Backtracking

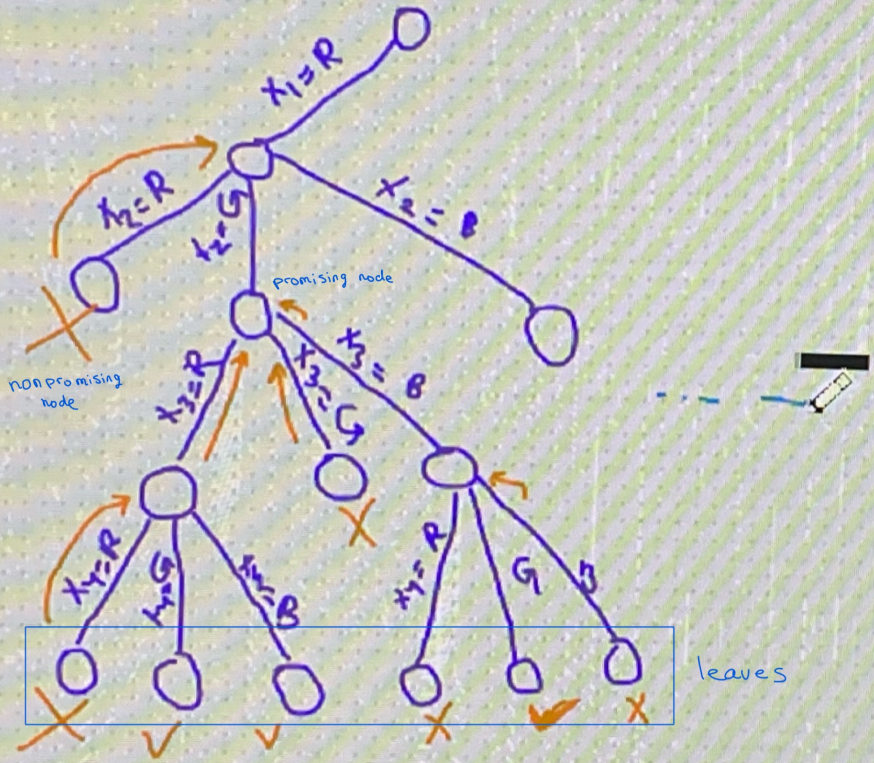
to find all possible solution

Conditions: no country could have adjacent color

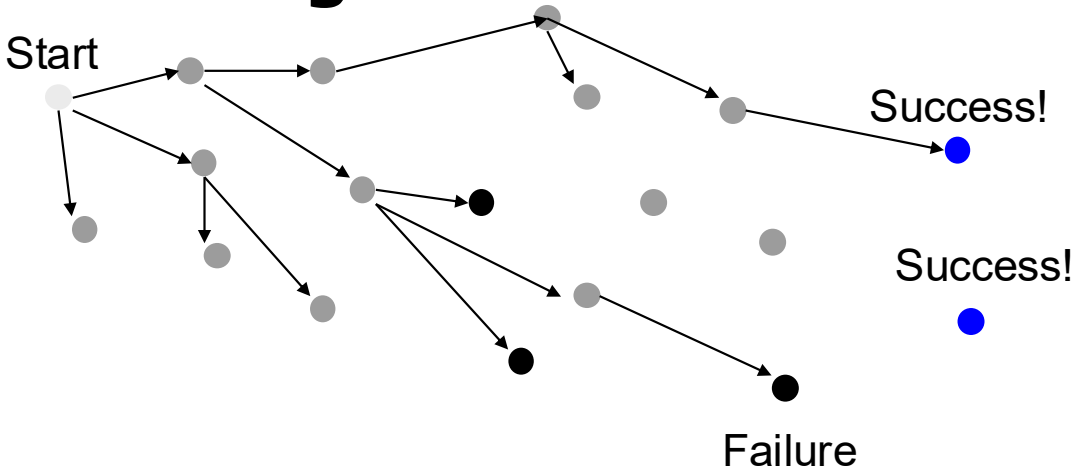




R, G, B



Backtracking



Problem space consists of states (nodes) and actions (paths that lead to new states). When in a node can can only see paths to connected nodes

If a node only leads to failure go back to its "parent" node. Try other alternatives. If these all lead to failure then more backtracking may be necessary.

Recursive Backtracking

Pseudo code for recursive backtracking algorithms

If at a solution, return success
for(every possible choice from current state / node)
  Make that choice and take one step along path
  Use recursion to solve the problem for the new node / state
  If the recursive call succeeds, report the success to the next high level
  Back out of the current choice to restore the state at the beginning of the loop.
Report failure



Backtracking

- Construct the state space tree:
  - *Root represents an initial state*
  - *Nodes reflect specific choices made for a solution's components.*
    - Promising and nonpromising nodes
    - leaves

- Explore the state space tree using depth-first search

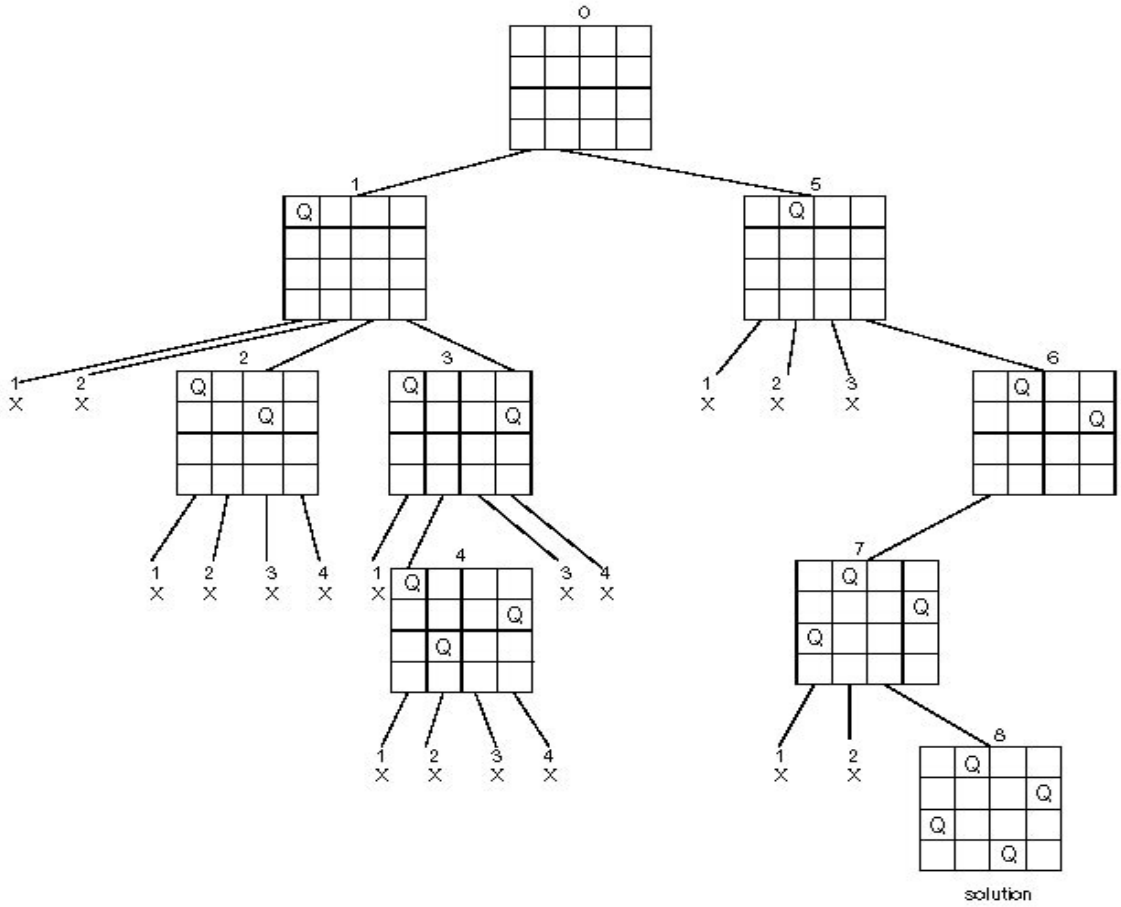
- "Prune" non-promising nodes
  - *dfs stops exploring subtree rooted at nodes leading to no solutions and...*
  - *"backtracks" to its parent node*

Example: The *n*-Queen problem

- Place *n* queens on an *n* by *n* chess board so that no two of them are on the same row, column, or diagonal



State Space Tree of the Four-queens Problem



The backtracking algorithm

- Backtracking is really quite simple--we "explore" each node, as follows:
- To "explore" node N:
  1. *If N is a goal node, return "success"*
  2. *If N is a leaf node, return "failure"*
  3. *For each child C of N,*
     3.1. Explore C
        3.1.1. If C was successful, return "success"
  4. *Return "failure"*



Exercises

- Continue the backtracking search for a solution to the four-queens problem to find the second solution to the problem.

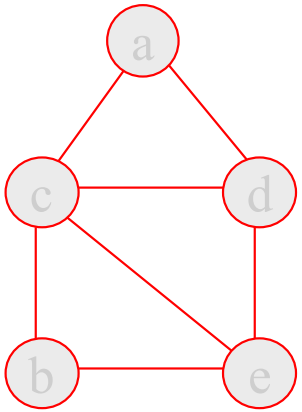
- A trick to use: the board is symmetric, obtain another solution by reflections.

- Get a solution to the 5-queens problem found by the back-tracking algorithm?
- Can you (quickly) find at least 3 other solutions?

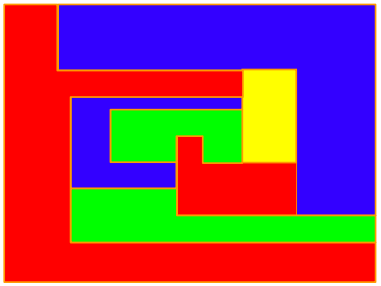


The *m*-Coloring Problem and Hamiltonian Problem

- 2-color
- 3-color
- Hamiltonian Circuit
  (use alphabet order to break the ties)



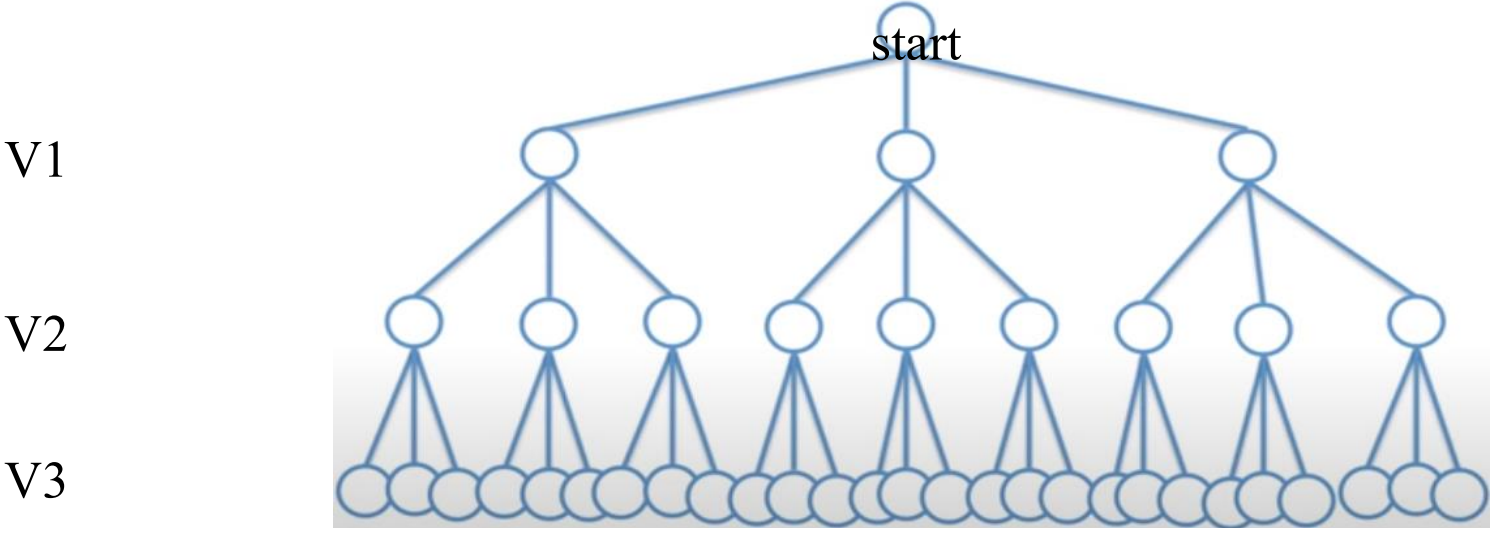
Graph Coloring
Map coloring



- m-coloring problem concerns finding all ways to color n regions on a map at most m different colors
- Adjacent regions must be in different colors
- You don't have enough information to choose colors
- Each choice leads to another set of choices
- One or more sequences of choices may (or may not) lead to a solution
- Many coloring problems can be solved with backtracking represented as undirected graph
  - *Regions are the nodes*
  - *Boundaries are the edges*

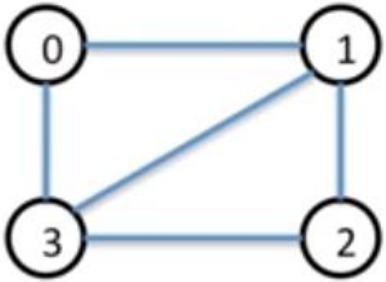
Graph Coloring
Map coloring

- The Color Theorem states that any map on a plane can be colored with no more than four colors, so that no two countries with a common border are the same color
- For regions n=3 and colors m=3, where there are no restrictions. Then, we get a large tree with m^n possible ways for coloring



Example 1 code in C++

- n=4, m=3 {red, green, blue}
- solution stored in array **x** holds the color for each node



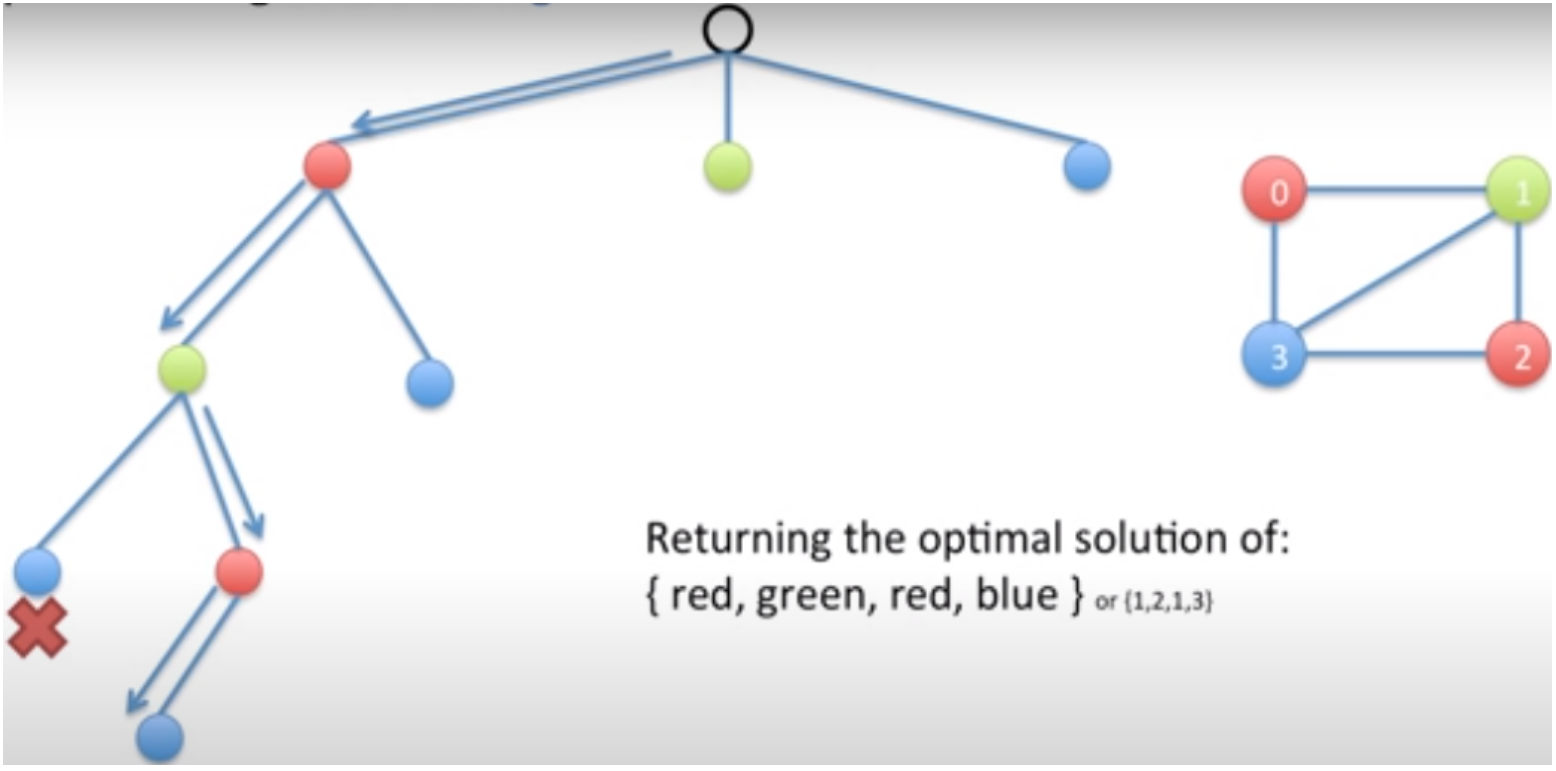
- Adjacency matrix G

n	0	1	2	3
0	0	1	0	1
1	1	0	1	1
2	0	1	0	1
3	1	1	1	0

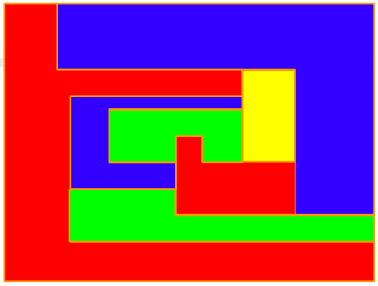
```
graphColour(int k){
   for(int c = 1; c<=m; c++){
      if(isSafe(k,c)){
        x[k] = c;
        if((k+1)<n)
           graphColour(k+1);
        else
            print x[]; return;
      }
   }
}
```

```
isSafe(int k, int c){
   for(int I = 0; i<n; i++){
      if(G[k][i] == 1 && c == x[i]{
         return false;
      }
   }
   return true;
}
```

Example cont.
State Space Tree

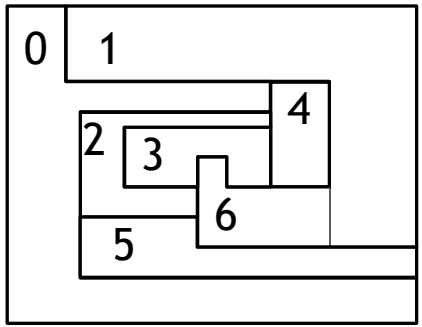


Example 2
complete Java code



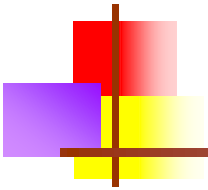
- You wish to color a map with not more than four colors
  - red, yellow, green, blue
- We need a data structure that is easy to work with, and supports:
  - Setting a color for each country
  - For each country, finding all adjacent countries
- We can do this with two arrays
  - An array of "colors", where `countryColor[i]` is the color of the ith country
  - A ragged array of adjacent countries, where `map[i][j]` is the jth country adjacent to country i
    - Example: `map[5][3]==8` means the 3th country adjacent to country 5 is country 8

Creating the map (adjacent nodes)




```
int map[][];

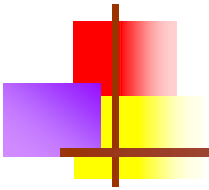
void createMap() {
    map = new int[7][];
    map[0] = new int[] { 1, 4, 2, 5 };
    map[1] = new int[] { 0, 4, 6, 5 };
    map[2] = new int[] { 0, 4, 3, 6, 5 };
    map[3] = new int[] { 2, 4, 6 };
    map[4] = new int[] { 0, 1, 6, 3, 2 };
    map[5] = new int[] { 2, 6, 1, 0 };
    map[6] = new int[] { 2, 3, 4, 1, 5 };
}
```



Setting the initial colors

```
static final int NONE = 0;
static final int RED = 1;
static final int YELLOW = 2;
static final int GREEN = 3;
static final int BLUE = 4;

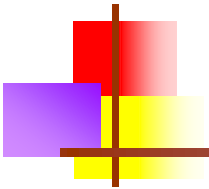
int mapColors[] = { NONE, NONE, NONE, NONE,
                    NONE, NONE, NONE };
```



The main program

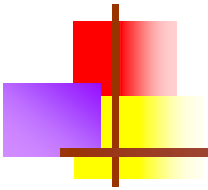
(The name of the enclosing class is ColoredMap)

```
public static void main(String args[]) {
        ColoredMap m = new ColoredMap();
        m.createMap();
        boolean result = m.explore(0, RED);
        System.out.println(result);
        m.printMap();
}
```



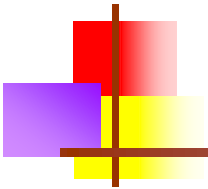
The backtracking method

```
boolean explore(int country, int color) {
    if (country >= map.length) return true; // goal is reached
    if (okToColor(country, color)) { // check if it is not a dead choice
        mapColors[country] = color; // incrementally built
        for (int i = RED; i <= BLUE; i++) {
            if (explore(country + 1, i)) return true; // explore new choice (country)
        }
    }
    return false; // backtracking
}
```



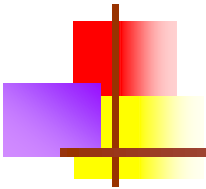
Checking if a color can be used

```
boolean okToColor(int country, int color) {
    for (int i = 0; i < map[country].length; i++) {
        int ithAdjCountry = map[country][i];
        if (mapColors[ithAdjCountry] == color) {
            return false;
        }
    }
    return true;
}
```



Printing the results

```
void printMap() {
    for (int i = 0; i < mapColors.length; i++) {
        System.out.print("map[" + i + "] is ");
        switch (mapColors[i]) {
            case NONE:   System.out.println("none");   break;
            case RED:    System.out.println("red");    break;
            case YELLOW: System.out.println("yellow"); break;
            case GREEN:  System.out.println("green");  break;
            case BLUE:   System.out.println("blue");   break;
        }
    }
}
```



Recap

- We went through all the countries recursively, starting with country zero
- At each country we had to decide a color
  - It had to be different from all adjacent countries
  - If we could not find a legal color, we reported failure
  - If we could find a color, used it and recurred with the next country
  - If we ran out of countries (colored them all), we reported success
- When we returned from the topmost call, we were done
- This algorithm is still $O(m^n)$. It is a known NP-hard problem but with backtracking we have a good representation

Comments

- *Backtracking provides the hope to solve some problem instances of nontrivial sizes by pruning non-promising branches of the state-space tree.*
- *The success of backtracking varies from problem to problem and from instance to instance.*
- *Backtracking possibly generates all possible candidates in an exponentially growing state-space tree.*



Other Backtracking Problems *

- 8 Queens
- Knight's Tour
- Knapsack problem / Exhaustive Search
  - *Filling a knapsack. Given a choice of items with various weights and a limited carrying capacity find the optimal load out. 50 lb. knapsack. items are 1 40 lb, 1 32 lb. 2 22 lbs, 1 15 lb, 1 5 lb. A greedy algorithm would choose the 40 lb item first. Then the 5 lb. Load out = 45lb. Exhaustive search 22 + 22 + 5 = 49.*



Branch and Bound



Branch and Bound (B& B)

- An enhancement of backtracking
  - ***Similarity***
    - A state space tree is used to solve a problem.
  - ***Difference***
    - The branch-and-bound algorithm does not limit us to any particular way of traversing the tree.
    - Used only for optimization problems (since the backtracking algorithm requires the using of DFS traversal and is used for non-optimization problems.

Branch and Bound

- The idea:

  Set up a bounding function, which is used to compute a bound (for the value of the objective function) at a node on a state-space tree and determine if it is promising
  - *Promising (if the bound is better than the value of the best solution so far): expand beyond the node.*
  - *Nonpromising (if the bound is no better than the value of the best solution so far): not expand beyond the node (pruning the state-space tree).*

Traveling Salesman Problem

An obvious way to construct the state-space tree

- A node = a vertex: a vertex in the graph.
- A node that is not a leaf represents all the tours that start with the path stored at that node; each leaf represents a tour (or non-promising node).
- Branch-and-bound: we need to determine a **lower** bound for each node
  - For example, to determine a lower bound for node [1, 2] means to determine a lower bound on the length of any tour that starts with edge 1—2.
- Expand each promising node, and stop when all the promising nodes have been expanded. During this procedure, prune all the nonpromising nodes.
  - Promising node: the node's lower bound is less than current minimum tour length.
  - Non-promising node: the node's lower bound is NO less than current minimum tour length.



Traveling Salesman Problem—Bounding Function 1

- Because a tour must leave every vertex exactly once, a lower bound on the length of a tour is the sum of the minimum (lower bound)cost of leaving every vertex.
  - *The lower bound on the cost of leaving vertex v_1 is given by the minimum of all the nonzero entries in row 1 of the adjacency matrix.*
  - *…*
  - *The lower bound on the cost of leaving vertex v_n is given by the minimum of all the nonzero entries in row n of the adjacency matrix.*
- Note: This is not to say that there is a tour with this length. Rather, it says that there can be no shorter tour.
- Assume that the tour starts with v1.



Traveling Salesman Problem—Bounding Function 2

- Because every vertex must be entered and exited exactly once, a lower bound on the length of a tour is the sum of the minimum cost of entering and leaving every vertex.
  - *For a given edge (u, v), think of half of its weight as the exiting cost of u, and half of its weight as the entering cost of v.*
  - *The total length of a tour = the total cost of visiting(entering and exiting) every vertex exactly once.*
  - *The lower bound of the length of a tour = the lower bound of the total cost of visiting (entering and exiting) every vertex exactly once.*
    - Calculation:
      - for each vertex, pick top two shortest adjacent edges (their sum divided by 2 is the lower bound of the total cost of entering and exiting the vertex);
      - add up these summations for all the vertices.
- Assume that the tour starts with vertex a and that b is visited before c.

Traveling salesman example 2

